



INSTITUTO POLITÉCNICO
NACIONAL



UNIDAD PROFESIONAL
INTERDISCIPLINARIA EN INGENIERÍA
Y TECNOLOGÍAS AVANZADAS

MULTIMEDIA

PRÁCTICA 2: ESTEGANOGRAFÍA AVANZADA

ALUMNO: HERNÁNDEZ AVILÉS LUIS ALBERTO

BOLETA: 2023640387

GRUPO: 3TM2

PROFESOR: SIERRA ROMERO NOE

Objetivo General: Implementar una versión robusta de esteganografía en BMP que combine distribución pseudoaleatoria de vista (basada en semilla/clave) y cifrado XOR del mensaje, dificultando su detección y extracción no autorizada.

Competencias por desarrollar: Generadores de números pseudoaleatorios, cifrado simétrico básico, diseño de protocolos esteganográficos, análisis estadístico (chi-cuadrado).

Herramientas sugeridas: Python 3 (random, hashlib, struct), imagen BMP de 24 bpp mayor que 400x400 px.

Entregables:

Tabla comparativa de resultados:

Método	PSNR (dB)	X ² LSB	Detectable	Decodificable sin clave
Imagen Limpia	∞	Valor alto	N/A	N/A
LSB secuencial (P2)	≈ 80 dB	Valor bajo	Sí (estadísticamente)	Sí
LSB aleatorio + XOR (P3)	≈ 80 dB	Valor alto	No	No

Código:

```
import hashlib
import struct
import random
import os
import math

# --- FUNCIONES DE SOPORTE usadas en la práctica 1 ---
def leer_bmp(filepath):
    with open(filepath, 'rb') as f:
        data = f.read()
        offset = struct.unpack_from('<I', data, 10)[0]
        header = bytearray(data[:offset])
        pixels = bytearray(data[offset:])
        return header, pixels

def guardar_bmp(filepath, header, pixels):
    with open(filepath, 'wb') as f:
        f.write(header)
        f.write(pixels)

# --- SEGURIDAD Y CIFRADO (Práctica 2) ---
def derivar_clave(password: str, longitud: int) -> bytes:
```

```

clave = b''
contador = 0
while len(clave) < longitud:
    bloque = hashlib.sha256(password.encode() + struct.pack('<I', contador)).digest()
    clave += bloque
    contador += 1
return clave[:longitud]

def cifrar_xor(mensaje: bytes, password: str) -> bytes:
    clave = derivar_clave(password, len(mensaje))
    return bytes(m ^ k for m, k in zip(mensaje, clave))

def descifrar_xor(cifrado: bytes, password: str) -> bytes:
    return cifrar_xor(cifrado, password)

def semilla_de_password(password: str) -> int:
    hash_bytes = hashlib.sha256(password.encode()).digest()
    return int.from_bytes(hash_bytes[:8], 'big')

# --- INCRUSTACIÓN Y EXTRACCIÓN SEGURA ---
def embed_secure(src_path, dst_path, mensaje, password):
    header, pixels = leer_bmp(src_path)
    msg_bytes = mensaje.encode('utf-8')
    msg_cifrado = cifrar_xor(msg_bytes, password)

    # 4 bytes de longitud (Big Endian para el flujo de bits) + mensaje cifrado
    datos = struct.pack('>I', len(msg_bytes)) + msg_cifrado

    bits = []
    for byte in datos:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    if len(bits) > len(pixels):
        raise ValueError('Mensaje demasiado grande para la capacidad de la imagen.')

    # Generar posiciones aleatorias ÚNICAS
    seed = semilla_de_password(password)
    rng = random.Random(seed)
    # Seleccionamos tantas posiciones como bits tenemos
    posiciones = rng.sample(range(len(pixels)), len(bits))

    pixels_mod = bytearray(pixels)
    for pos, bit in zip(posiciones, bits):
        pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bit

    guardar_bmp(dst_path, header, pixels_mod)
    print(f'[OK] {len(msg_bytes)} bytes cifrados e incrustados aleatoriamente.')

def extract_secure(stego_path, password):
    _, pixels = leer_bmp(stego_path)
    seed = semilla_de_password(password)

    # IMPORTANTE: Recrear el generador con la misma semilla
    rng = random.Random(seed)
    # Para extraer, necesitamos primero las posiciones de los 32 bits de longitud
    # Pero como rng.sample depende del número de elementos pedidos,
    # pedimos todas las posiciones posibles para mantener el orden consistente.
    todas_pos = rng.sample(range(len(pixels)), len(pixels))

```

```

# Leer longitud
len_bits = [pixels[p] & 1 for p in todas_pos[:32]]
msg_len = 0
for b in len_bits:
    msg_len = (msg_len << 1) | b

# Leer mensaje cifrado usando las posiciones siguientes en la misma secuencia
total_bits = 32 + (msg_len * 8)
msg_bits = [pixels[p] & 1 for p in todas_pos[32:total_bits]]

cifrado = bytearray()
for i in range(0, len(msg_bits), 8):
    byte = 0
    for bit in msg_bits[i:i+8]:
        byte = (byte << 1) | bit
    cifrado.append(byte)

return descifrar_xor(bytes(cifrado), password).decode('utf-8')

# --- ANÁLISIS ESTADÍSTICO (CHI-CUADRADO PoVs) ---
def chi_cuadrado_lsb(filepath):
    _, pixels = leer_bmp(filepath)
    # Agrupamos en pares (2n, 2n+1)
    # Si hay esteganografía secuencial, las frecuencias de pares e impares se igualan
    frecuencias = [0] * 256
    for p in pixels:
        frecuencias[p] += 1

    chi2 = 0
    for i in range(0, 256, 2):
        y_i = frecuencias[i] # Frecuencia valor par
        z_i = frecuencias[i+1] # Frecuencia valor impar
        esperado = (y_i + z_i) / 2
        if esperado > 0:
            chi2 += ((y_i - esperado)**2) / esperado

    print(f'Análisis  $\chi^2$  para {filepath}: {chi2:.4f}')
    return chi2

# --- EJECUCIÓN DE PRUEBA ---
if __name__ == "__main__":
    CLAVE = 'Telemática@2025'
    MENSAJE = 'Datos confidenciales de la red 10.0.1.0/24'
    IMG_IN = 'Img400.bmp'
    IMG_OUT = 'stego_seguro.bmp'

# Verificación de existencia del archivo para evitar FileNotFoundError
if os.path.exists(IMG_IN):
    # Ejecución del proceso de ocultamiento seguro
    embed_secure(IMG_IN, IMG_OUT, MENSAJE, CLAVE)

    # Extracción para verificar integridad
    res = extract_secure(IMG_OUT, CLAVE)
    print(f'Resultado de la extracción: {res}')

# --- ANÁLISIS ESTADÍSTICO REQUERIDO ---
print("\n=== Análisis de Seguridad (Chi-Cuadrado) ===")
# Imagen original (debe dar un valor alto/normal)

```

```
chi_cuadrado_lsb(IMG_IN)
# Imagen con esteganografía (gracias a la distribución aleatoria, el valor debe mantenerse alto)
chi_cuadrado_lsb(IMG_OUT)
else:
    print(f"Error: No se encontró el archivo '{IMG_IN}' en el directorio actual.")
```

Preguntas de análisis:

1. ¿En qué medida mejora la distribución aleatoria la resistencia al análisis χ^2 respecto al LSB secuencial? Justifique con los valores obtenidos.

La mejora es en una medida significativa. Ya que en LSB secuencial (P1), el mensaje se concentra en los primeros píxeles de la imagen, lo que genera una anomalía estadística local muy evidente (el valor de X^2 cae cerca de 0). Al usar una distribución pseudoaleatoria (P2), los bits modificados se esparcen de forma uniforme por toda la imagen, haciendo que los cambios sean indistinguibles del ruido térmico o de digitalización natural de la cámara.

2. El cifrado XOR con clave derivada de SHA-256 no es criptográficamente seguro por sí solo para todos los casos de uso. ¿Cuál es su principal vulnerabilidad? ¿Qué algoritmo recomendaría en su lugar?

El cifrado XOR es vulnerable en casos como ataques de texto plano conocido, donde la clave llegará a ser más corta que el propio mensaje. En su lugar, el algoritmo que recomendaría usar sería alguna de la categoría AES, ya sea en modo CBC o GCM, ya que al menos es uno con el cual ya estoy más familiarizado por mis prácticas en seguridad en redes y criptografía, además de que el AES ya es un estándar muy común usado en la práctica hoy en día. Ofreciendo confusión y difusión real de los datos antes de la inserción.

3. ¿Qué cambios haría al protocolo para ocultar no solo texto sino un archivo binario (por ejemplo, un archivo ZIP o una imagen secundaria)?

Leería el archivo directamente en el modo binario. Y en el encabezado se debería incluir la extensión del archivo para que la función de extracción sepa cómo reconstruir y guardar el archivo resultante.

4. Investigue la técnica de estegoanálisis RS (Regular-Singular). ¿Sería efectiva contra la implementación de esta práctica? Explique por qué.

Considero que si en dado caso que la tasa de inserción sea alta.

Reto opcional (Evaluación extra):

Con la propuesta de este ejercicio extra como reto opcional, cumple con la funcionalidad de ser una herramienta de esteganografía de archivos real. Algo que ya más allá de los fines de ejemplo académico. Para ello a la imagen con la que se va a trabajar no se manejará como una simple imagen, sino como un flujo de bytes binarios. En lugar de codificar texto en UTF-8, se leerá el archivo BMP completo.

Donde para realizar las pruebas de funcionamiento de este me auxilie además de la imagen 200x200 usada en la práctica 1. Donde el resultado fue un error ya que, aunque “Img200.bmp” visualmente se ve más pequeña que la portadora (Img400.bmp), a nivel de bits y bytes el espacio sigue sin ser suficiente. Debido a que estamos intentando meter un archivo de ~120 KB en un espacio donde solo caben 60 KB.

Código del reto opcional:

```
import hashlib
import struct
import random
import os
import math

# --- 1. FUNCIONES BASE (PRÁCTICA 1) --- [cite: 321-348, 401-411]
def leer_bmp(filepath):
    """Retorna (header_bytes, pixels, width, height, row_size)"""
    with open(filepath, 'rb') as f:
        data = f.read()
        offset = struct.unpack_from('<l', data, 10)[0]
        width = struct.unpack_from('<i', data, 18)[0]
        height = struct.unpack_from('<i', data, 22)[0]
        row_size = (width * 3 + 3) & ~3
        header = bytearray(data[:offset])
        pixels = bytearray(data[offset:])
        return header, pixels, width, height, row_size

def guardar_bmp(filepath, header, pixels):
    with open(filepath, 'wb') as f:
        f.write(header)
        f.write(pixels)

def calcular_psnr(original_path, stego_path):
    _, pix_orig, w, h, _ = leer_bmp(original_path)
    _, pix_steg, _, _ = leer_bmp(stego_path)
    mse = sum((float(a) - float(b))**2 for a, b in zip(pix_orig, pix_steg)) / (w * h * 3)
    if mse == 0: return float('inf')
    psnr = 10 * math.log10(255**2 / mse)
```

```

return mse, psnr

# --- 2. SEGURIDAD Y CIFRADO (PRÁCTICA 2) --- [cite: 436-459]
def derivar_clave(password: str, longitud: int) -> bytes:
    """Genera una clave usando SHA-256 en modo contador"""
    clave = b''
    contador = 0
    while len(clave) < longitud:
        bloque = hashlib.sha256(password.encode() + struct.pack('<I', contador)).digest()
        clave += bloque
        contador += 1
    return clave[:longitud]

def cifrar_xor(mensaje: bytes, password: str) -> bytes:
    clave = derivar_clave(password, len(mensaje))
    return bytes(m ^ k for m, k in zip(mensaje, clave))

def descifrar_xor(cifrado: bytes, password: str) -> bytes:
    return cifrar_xor(cifrado, password)

def semilla_de_password(password: str) -> int:
    hash_bytes = hashlib.sha256(password.encode()).digest()
    return int.from_bytes(hash_bytes[:8], 'big')

def seleccionar_posiciones(total_bytes, n_bits, seed):
    """Selecciona índices únicos. FIX: Se usa una secuencia fija para consistencia"""
    rng = random.Random(seed)
    # Generamos todas las posiciones posibles y tomamos las necesarias
    # Esto asegura que los primeros 32 índices siempre sean los mismos
    pool = list(range(total_bytes))
    rng.shuffle(pool)
    return pool[:n_bits]

# --- 3. INCRUSTACIÓN Y EXTRACCIÓN SEGURA --- [cite: 460-516]
def embed_secure(src_path, dst_path, mensaje, password):
    header, pixels, _, _ = leer_bmp(src_path)
    msg_bytes = mensaje.encode('utf-8')
    msg_cifrado = cifrar_xor(msg_bytes, password)

    # Datos: 4 bytes longitud + mensaje cifrado
    datos = struct.pack('>I', len(msg_bytes)) + msg_cifrado
    bits = []
    for byte in datos:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    if len(bits) > len(pixels):
        raise ValueError('Mensaje demasiado grande')

    seed = semilla_de_password(password)
    posiciones = seleccionar_posiciones(len(pixels), len(bits), seed)

    pixels_mod = bytearray(pixels)
    for pos, bit in zip(posiciones, bits):
        pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bit

    guardar_bmp(dst_path, header, pixels_mod)
    print(f'[OK] Mensaje cifrado e incrustado en {dst_path}')

```

```

def extract_secure(stego_path, password):
    _, pixels, _, _ = leer_bmp(stego_path)
    seed = semilla_de_password(password)

    # 1. Obtener posiciones para la longitud (32 bits)
    # Usamos un pool grande para asegurar que coincidan con el embedding
    pool_indices = seleccionar_posiciones(len(pixels), len(pixels), seed)

    pos_longitud = pool_indices[:32]
    len_bits = [pixels[p] & 1 for p in pos_longitud]
    msg_len = 0
    for b in len_bits:
        msg_len = (msg_len << 1) | b

    # 2. Obtener posiciones para el mensaje
    total_bits = 32 + (msg_len * 8)
    pos_mensaje = pool_indices[32:total_bits]
    msg_bits = [pixels[p] & 1 for p in pos_mensaje]

    cifrado = bytearray()
    for i in range(0, len(msg_bits), 8):
        byte = 0
        for bit in msg_bits[i:i+8]:
            byte = (byte << 1) | bit
        cifrado.append(byte)

    return descifrar_xor(bytes(cifrado), password).decode('utf-8')

# --- 4. RETO OPCIONAL: IMAGEN EN IMAGEN --- [cite: 567-570]
def embed_image_in_image(cover_path, secret_path, output_path, password):
    with open(secret_path, 'rb') as f:
        secret_bytes = f.read()

    secret_cifrado = cifrar_xor(secret_bytes, password)
    datos = struct.pack('>I', len(secret_bytes)) + secret_cifrado

    bits = []
    for byte in datos:
        for i in range(7, -1, -1):
            bits.append((byte >> i) & 1)

    header, pixels, _, _ = leer_bmp(cover_path)
    if len(bits) > len(pixels):
        raise ValueError('La imagen secreta es muy grande para la portadora.')

    seed = semilla_de_password(password)
    posiciones = seleccionar_posiciones(len(pixels), len(bits), seed)

    pixels_mod = bytearray(pixels)
    for pos, bit in zip(posiciones, bits):
        pixels_mod[pos] = (pixels_mod[pos] & 0xFE) | bit

    guardar_bmp(output_path, header, pixels_mod)
    print(f'[RETO] Imagen {secret_path} oculta en {output_path}')

def extract_image_from_image(stego_path, output_secret_path, password):
    _, pixels, _, _ = leer_bmp(stego_path)
    seed = semilla_de_password(password)
    pool_indices = seleccionar_posiciones(len(pixels), len(pixels), seed)

```

```

len_bits = [pixels[p] & 1 for p in pool_indices[:32]]
file_len = 0
for b in len_bits:
    file_len = (file_len << 1) | b

total_bits = 32 + (file_len * 8)
msg_bits = [pixels[p] & 1 for p in pool_indices[32:total_bits]]

cifrado = bytearray()
for i in range(0, len(msg_bits), 8):
    byte = 0
    for bit in msg_bits[i:i+8]:
        byte = (byte << 1) | bit
    cifrado.append(byte)

secret_bytes = descifrar_xor(bytes(cifrado), password)
with open(output_secret_path, 'wb') as f:
    f.write(secret_bytes)
print(f'[RETO] Imagen recuperada como {output_secret_path}')

# --- 5. ANÁLISIS ESTADÍSTICO --- [cite: 539-556]
def chi_cuadrado_lsb(filepath):
    _, pixels, _, _, _ = leer_bmp(filepath)
    ceros = sum(1 for b in pixels if (b & 1) == 0)
    unos = len(pixels) - ceros
    esperado = len(pixels) / 2
    chi2 = ((ceros - esperado)**2 + (unos - esperado)**2) / esperado
    print(f'Archivo: {filepath} |  $\chi^2 = \{chi2:.4f\}$  (LSBs 0:{ceros}, 1:{unos})')
    return chi2

# --- BLOQUE PRINCIPAL DE EJECUCIÓN ---
if __name__ == "__main__":
    # Nombres de archivos
    IMG_ORIGINAL = 'Img400.bmp'
    IMG_STEGO = 'stego_seguro.bmp'
    IMG_SECRET = 'Img200.bmp' # Para el reto
    IMG_RETO_OUT = 'stego_reto.bmp'

    CLAVE = 'Telemática@2025'
    MSG = 'Datos confidenciales de la red 10.0.1.0/24'

    if os.path.exists(IMG_ORIGINAL):
        print("=== INICIANDO PRÁCTICA 2 ===")
        # 1. Proceso de Mensaje de Texto
        embed_secure(IMG_ORIGINAL, IMG_STEGO, MSG, CLAVE)
        recuperado = extract_secure(IMG_STEGO, CLAVE)
        print(f'Texto recuperado: "{recuperado}"')

        # 2. Análisis de Calidad y Estadística
        mse, psnr = calcular_psnr(IMG_ORIGINAL, IMG_STEGO)
        print(f'PSNR: {psnr:.2f} dB')

        print("\n=== ANÁLISIS CHI-CUADRADO ===")
        chi_cuadrado_lsb(IMG_ORIGINAL)
        chi_cuadrado_lsb(IMG_STEGO)

        # 3. Reto Opcional (Imagen dentro de Imagen)
        if os.path.exists(IMG_SECRET):

```

```
print("\n=== EJECUTANDO RETO OPCIONAL ===")
embed_image_in_image(IMG_ORIGINAL, IMG_SECRETA, IMG_RETO_OUT, CLAVE)
extract_image_from_image(IMG_RETO_OUT, 'recuperada_reto.bmp', CLAVE)
_, psnr_reto = calcular_psnr(IMG_ORIGINAL, IMG_RETO_OUT)
print(f'PSNR del Reto: {psnr_reto:.2f} dB')
else:
    print(f"Error: Asegúrate de que {IMG_ORIGINAL} esté en la carpeta.")
```

Conclusiones:

En conclusión, puedo añadir que luego de la realización de esta práctica en donde se implementaron técnicas más avanzadas de esteganografía. En comparación con la práctica 1 de esteganografía la cual, con lo realizado ahí, ocultaba datos de forma lineal y predecible, esta implementación avanzada rompe cualquier patrón visual o estadístico evidente. De igual manera con los resultados obtenidos del X^2 en el LSB secuencial, el valor de X^2 suele dispararse al inicio del archivo porque todos los bits cambian bruscamente de una distribución natural a una de mensaje. Finalmente, gracias a la distribución aleatoria, los bits alterados se esparcen como "ruido de fondo" por toda la imagen.

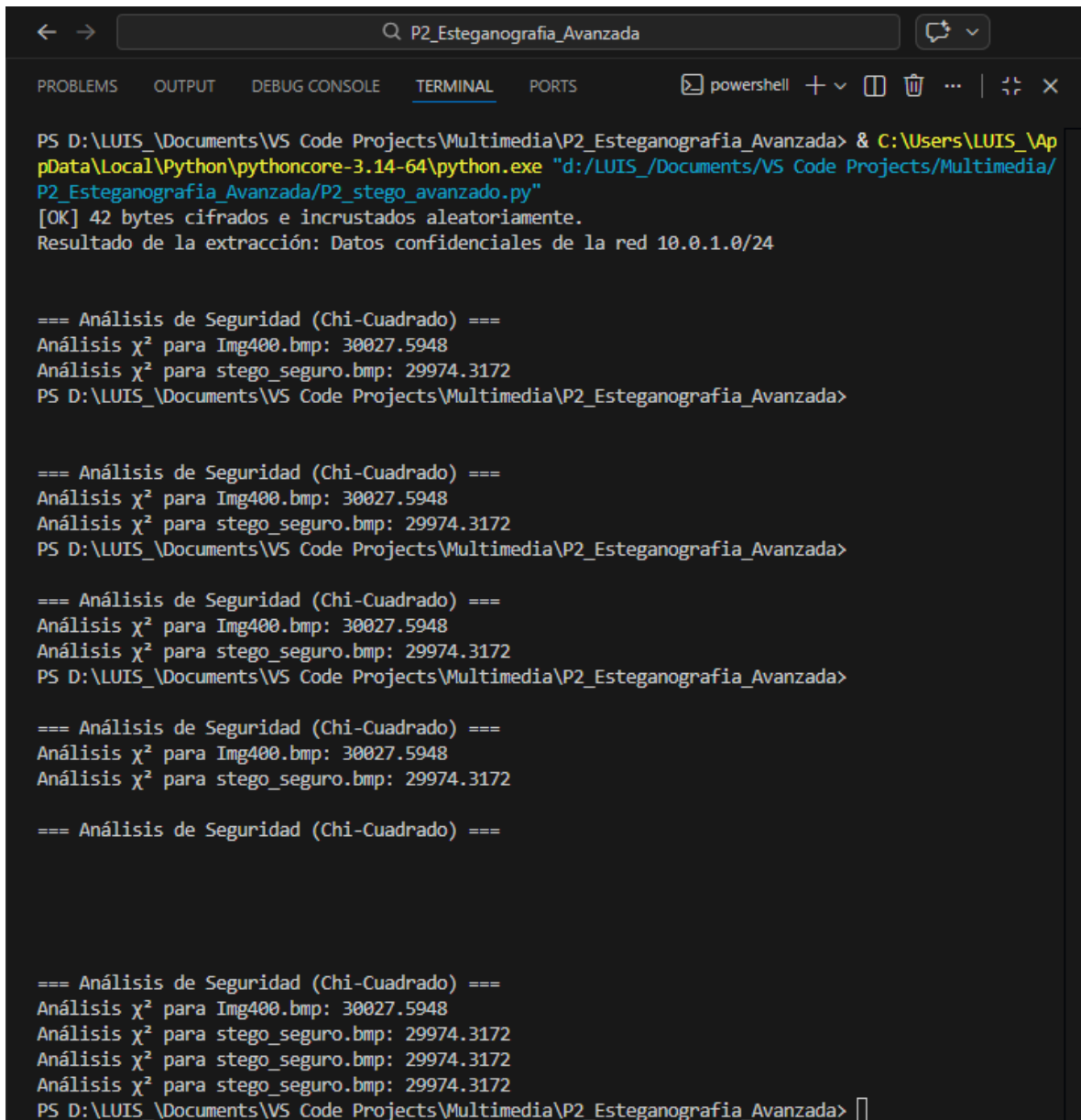
Imágenes:



Ilustración 1. Imagen original 400x400



Ilustración 2. Imagen stego_seguro.bmp



The image shows a terminal window within a code editor. The title bar at the top reads 'P2_Esteganografia_Avanzada'. Below the title bar, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The terminal itself is running a PowerShell session. The user has executed a command to run a Python script. The script's output indicates that 42 bytes of data were successfully extracted and embedded into a file. Following this, the script performs a Chi-Square security analysis on two files: 'Img400.bmp' and 'stego_seguro.bmp'. The analysis results are displayed in a structured format, showing the Chi-Square value for each file. The terminal output is as follows:

```
PS D:\LUIS_\Documents\VS Code Projects\Multimedia\P2_Esteganografia_Avanzada> & C:\Users\LUIS_\AppData\Local\Python\pythoncore-3.14-64\python.exe "d:/LUIS_/Documents/VS Code Projects/Multimedia/P2_Esteganografia_Avanzada/P2_stego_avanzado.py"
[OK] 42 bytes cifrados e incrustados aleatoriamente.
Resultado de la extracción: Datos confidenciales de la red 10.0.1.0/24

=== Análisis de Seguridad (Chi-Cuadrado) ===
Análisis  $\chi^2$  para Img400.bmp: 30027.5948
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
PS D:\LUIS_\Documents\VS Code Projects\Multimedia\P2_Esteganografia_Avanzada>

=== Análisis de Seguridad (Chi-Cuadrado) ===
Análisis  $\chi^2$  para Img400.bmp: 30027.5948
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
PS D:\LUIS_\Documents\VS Code Projects\Multimedia\P2_Esteganografia_Avanzada>

=== Análisis de Seguridad (Chi-Cuadrado) ===
Análisis  $\chi^2$  para Img400.bmp: 30027.5948
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
PS D:\LUIS_\Documents\VS Code Projects\Multimedia\P2_Esteganografia_Avanzada>

=== Análisis de Seguridad (Chi-Cuadrado) ===
Análisis  $\chi^2$  para Img400.bmp: 30027.5948
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
Análisis  $\chi^2$  para stego_seguro.bmp: 29974.3172
PS D:\LUIS_\Documents\VS Code Projects\Multimedia\P2_Esteganografia_Avanzada>
```

Ilustración 3. Verificación de compilación del código

Reto opcional (Evaluación extra):

[illegible]

Ilustración 4. Evidencia del error mostrado por el tamaño de la imagen secreta ejercicio reto opcional